University of Glasgow | School of Computing Science

Mini Project

Course Name	**Programming Methodology (PM)/Computer Organisation & Architecture (COA)**
Coursework Number	**1**
Deadline	**Time: 23:59 Date: [25 Nov 2026] (Week 13)**
% Contribution to final course mark	**35% (CSC1103) and 30% (CSC1104)**
Solo or Group	**Group ✓**
Anticipated Hours	**15 per group member**
Submission Instructions	**1. Submit via SIT_Xsite a Zip file containing your source code directory (UoG-PMCOA.zip), one PMCOAReportGroupxx.pdf file plus group presentation video/Demo video** **2. Declaration Form** **3. Peer Evaluation Form submitted separately**
Please Note: This Coursework cannot be Re-Done	

Code of Assessment Rules for Coursework Submission

Deadlines for the submission of coursework which is to be formally assessed will be published in course documentation, and work which is submitted later than the deadline will be subject to penalty as set out below.

The primary grade and marks awarded for coursework which is submitted after the published deadline will be calculated as follows:

(i) in respect of work submitted not more than four working days after the deadline

- the work will be assessed in the usual way;
- the primary grade and mark so determined will then be reduced by 15% for each working day (or part of a working day) the work was submitted late.

(ii) work submitted more than four working days after the deadline will be awarded Grade F.

Penalties for late submission of coursework will not be imposed if good cause is established for the late submission. You should submit documents supporting good cause to Admin-In-Charge.

Penalty for non-adherence to Submission Instructions is 2 bands
You must complete an "Own Work" form

Programming Methodology (UoG-PM) & Computer Organisation & Architecture (COA) 2026-27

Mini Project: Tick-to-Trade on the Edge

Low-Latency Market Data Engine in C on Raspberry Pi 5

Introduction

The goal of this Mini Project is to familiarise yourselves with the design, implementation and performance testing of C programming on a real edge computing device such as personal laptop and Raspberry Pi 5. You will learn why C remains the language of choice wherever speed, memory and power are limited: in embedded systems, robotics, networking and high-frequency trading (HFT).

The project is released in Week 5 and built in five stages that follow the lectures. Each stage uses only the C you have just learnt, so your program grows as your knowledge grows: from simple loops, to arrays, pointers, strings, structs and finally dynamic memory.

Given the problem statement, you will be required to go through the whole cycle of problem definition, problem analysis and pseudocode design. You will then implement, test, measure and demonstrate your market data engine in C on both laptop and Pi, including a visible signal on the Pi's onboard status LED.

This project rewards understanding over code volume. Generative AI can write a moving average function; it cannot explain why your program became faster when you changed how data sits in memory. Your marks depend on measuring, predicting and explaining your program's behaviour on real hardware.

Assessed Task

Assume you are the embedded C engineer for a financial technology start-up. The company wants a low-cost, low-power "signal box" built on a Raspberry Pi 5. It must read a stream of real market trades, compute trading indicators, and raise a BUY or SELL signal on its onboard status light and screen, faster than the market moves.

In HFT, every microsecond has a price: a program that falls behind the market is trading on old information. Your team must therefore prove, with measurements taken on the Pi, that your program is both correct and fast.

The Dataset

The dataset is one day of real cryptocurrency trades from Binance's free public market data (https://data.binance.vision). It is given in the XSite mini project folder as three files:

File	Content	Same for all groups?	Purpose
test20.csv	20 consecutive trades from your group's assigned position in the day, with expected answers	No, unique per group	Correctness checking
dev_1M.csv	1 million consecutive trades from your group's assigned slice of the day	No, unique per group	Development and debugging
stress_full.csv	The full day (several million trades)	Yes, identical for all groups	Final benchmarking and marked measurements

All three files come from the same trading symbol on the same day. Every group benchmarks on the identical stress_full.csv, so timings and speedups are directly comparable across groups. Each group is also assigned its own moving-average window sizes (e.g. 20/50, 30/100), so no two groups produce the same results. For test20.csv only, every group uses short = 3 and long = 5, so the expected answers can be checked by hand; your group's own window sizes apply to dev_1M.csv and stress_full.csv.

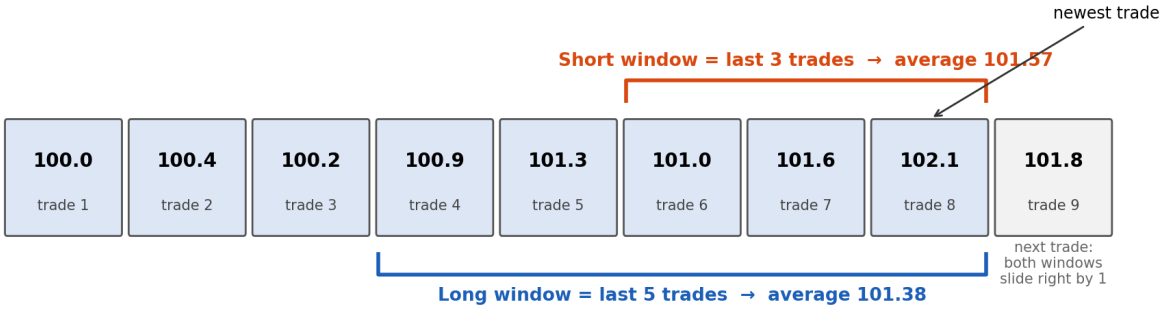
Your group's window sizes are listed below, and in the file group_info.txt in your group's folder:

Group	Short SMA window (trades)	Long SMA window (trades)
Group 01	10	60
Group 02	15	60
Group 03	20	60
Group 04	25	60
Group 05	30	60
Group 06	35	60
Group 07	40	60
Group 08	10	100
Group 09	15	100
Group 10	20	100
Group 11	25	100
Group 12	30	100
Group 13	35	100
Group 14	40	100
Group 15	10	200
Group 16	15	200
Group 17	20	200
Group 18	25	200
Group 19	30	200
Group 20	35	200
Group 21	40	200

What Are the Short and Long Windows?

SMA stands for Simple Moving Average: the plain average price of the most recent trades, where every price counts equally. The window is how many recent trades are averaged. Each time a new trade arrives, the window slides forward by one: the newest trade joins and the oldest one leaves (Figure 1). The exact formulas are in the section “The Maths You Need”.

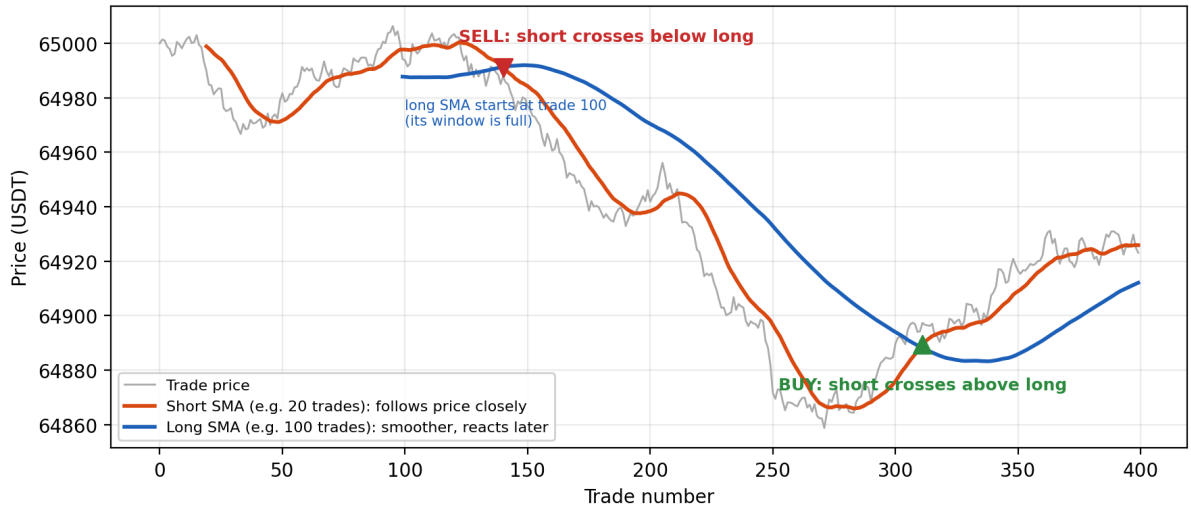
Figure 1: Short and long windows over the most recent trades (test windows 3 and 5)



- **Short window** (a few trades, e.g. 3 in test20 or 20 in your group's pair): follows the price closely and reacts quickly, but jumps around with every small move.
- **Long window** (more trades, e.g. 5 in test20 or 100 in your group's pair): smoother and shows the overall trend, but reacts more slowly.

Because the short average reacts first, it crosses the long average when the trend changes. When the short average rises above the long average, prices have started going up: a BUY signal. When it falls below, prices have started going down: a SELL signal (Figure 2). A moving average only has a value once its window is full: the long average in Figure 2 starts at trade 100.

Figure 2: Short SMA crossing the long SMA gives BUY and SELL signals (illustration)



Note : Figure 2 uses made-up prices for illustration. With your own data and window sizes, the lines and signals will look different.

Each row holds one trade with seven comma-separated fields:

n	Field	Meaning
1	trade Id	Sequential trade number
2	price	Price of the trade, up to 8 decimal places
3	qty	Quantity traded
4	quoteQty	price × qty (use it to check your own multiplication)
5	time	Timestamp in microseconds since 1 Jan 1970
6	isBuyerMaker	True: a seller started the trade (count it as a SELL). False: a buyer started it (count it as a BUY)
7	isBestMatch	True/False (not used)

Project Stages

The main task of the team is to develop the market data engine in the C programming language on both laptop and the Raspberry Pi 5, in the following five stages. Keep the working program from every stage: each one becomes a rung on your efficiency ladder.

Week	Lecture topic	Project stage	Lab checkpoint (formative)
5–6	Recursive functions	Stage 0: Getting started on the Pi	Week 6: program runs on Pi; LED blinks
6–7	Arrays	Stage A: Streaming engine	Week 7: matches test20.csv; signal plot drawn
8–9	Pointers, strings	Stage A+: Own reader and fast parser	Week 9: own parser matches test20.csv
10–11	Structs	Stage B: In-memory engine and efficiency ladder	Week 11: ladder with at least four rungs
12–13	Dynamic memory	Stage C: Real-time replay, signal and the cost of slow code	Week 13: hand-in and live demo(if schedule permits)

Lab checkpoints are not marked. They are pointers that can help you before you fall behind.

Stage 0: Getting Started on the Pi (Weeks 5–6)

You will write and test your code on your own Windows or Mac laptop first, using VS Code (for CSC1103 delivery). The provided timing.h and led.h work on Windows, Mac and the Pi (on a laptop, the LED is printed on screen instead). All timings in your logbook and report must be measured on both laptop (CSC1103 delivery) and Pi (CSC1104 delivery).

For CSC 1103 delivery

1) Compile and run a C program on your laptop from the terminal with gcc.
    a) Write a loop that adds up the numbers 1 to N, where N is typed in by the user. Start with N = 100,000,000. Time it with the provided now_ns() function at -O0, -O1, -O2 and -O3, and record the speedups.

    Why 100 million? Each addition takes about a nanosecond, so a small loop of, say, 1,000 numbers finishes in about a microsecond: too fast to time reliably, because reading the clock itself takes tens of nanoseconds. With 100 million additions the loop takes a fraction of a second, which you can measure accurately and repeat. It is also close to the scale of the real work: millions of trades.

    Watch the optimiser. Always print the sum. If the result is never used, or N is fixed in the code, the compiler at -O2 or -O3 may work out the answer without running the loop at all, giving an impossibly fast time. If that happens, record it in your logbook and explain it.

    b) Write a recursive and an iterative version of the same sum (1 + 2 + … + N, with N = 10,000, repeated many times). Time both and explain the difference using what you know about function calls and the stack. The depth is kept at 10,000 because much deeper recursion can overflow the stack and crash the program.

    c) Run the provided test_helpers.c to check your set-up. Then, in your own program, turn the green LED on just before your timed loop starts and off when it ends (using led_init(), led_on(), led_off() and led_cleanup()), so you can see the speedup between -O0 and -O2 as well as measure it.

    d) Start your prediction logbook: predict every result before you measure it.

For CSC 1104 delivery

1) Repeat Step 1 for Pi 5 for CSC1104 delivery. Log in to your Pi over SSH, then compile and run a C program from the terminal with gcc. Also, on the Pi, run the program with sudo so it can control the LED.
2) Pi comparison. Run your Stage 0 program on the Pi and on at least one group member's laptop (Windows or Mac), at -O0 and -O2. Record both sets of timings in one logbook entry, and explain the differences you see. Only the Pi's timings count towards your marks; the laptop comparison is marked on the quality of your explanation.

Stage A: Streaming Engine (Weeks 6–7)

Use the provided helper library as reference (you can create yours) to read one trade at a time for both Mac/Windows (CSC1103) and Pi (CSC1104). No pointers or structs are needed:

```
open_trades("dev_1M.csv");
while (next_trade()) {
    double p = trade_price();
    double q = trade_qty();
    /* update VWAP sums, OHLC bar, SMA windows, signal */
}
close_trades();
```

1) Compute the volume-weighted average price (VWAP) using two running sums.
2) Compute one-second OHLC (open, high, low, close) bars using four variables that reset every new second.
3) Compute a short and a long simple moving average (SMA), using an array as a circular window of the most recent prices.
4) Raise a BUY signal when the short SMA crosses above the long SMA, and a SELL signal when it crosses below. Count the signals.
5) **Plot your signals.** For a slice of about 2,000 trades from dev_1M.csv that contains at least one BUY and one SELL, write each trade to a text file (signals.dat), then draw it with simple gnuplot in C (http://www.gnuplot.info) . Check on the plot that every BUY and SELL sits exactly where the two SMA lines cross. Include the plot (signals.png) in your report.
6) Your output must match the expected answers for test20.csv, using the test windows short = 3 and long = 5.

Stage A+: Own Reader and Fast Parser (Weeks 8–9)

For both CSC 1103 delivery (laptop) and CSC1104 delivery (Pi)

1) **Week 8 (pointers).** Replace the helper library with your own function that hands back several results through pointer parameters. Study the helper library's source to see how fopen and fgets can be used:

```
int read_trade(FILE *fp, long long *ts, double *price,
               double *qty, int *is_buy);
```

2) **Week 9 (strings).** The helper library uses sscanf, which is slow. Write a faster parser that walks each line as a character array and converts the digits yourself. Measure the speedup on stress_full.csv.

Stage B: In-Memory Engine and Efficiency Ladder (Weeks 10–11)

For both CSC 1103 delivery (laptop) and CSC1104 delivery (Pi)

1) Define a Trade struct and load every trade into a fixed-size global array of structs (e.g. static Trade trades[MAX_TRADES];). A global array sits outside the stack, so millions of trades fit.

2) Compute the SMA twice: first naively, by re-adding the whole window at every trade, then with a running sum. Compare the times.
3) Store prices as fixed-point integers (long long) instead of double, and compare.
4) Build your efficiency ladder: a table and graph of the speedup of every version against the your Version 0 program. Include at least four of these rungs:
   - compiler optimisation flags (-O0, -O2, -O3) & -mcpu=cortex-a76 (for Pi only);
   - your own fast parser in place of the helper library;
   - a running-sum SMA in place of recomputing the whole window;
   - fixed-point integer prices in place of double;
   - processing from memory in place of re-reading the file;
   - (Week 12) memory allocated with malloc to fit the data exactly.

Stage C: Real-Time Replay, Signal and the Cost of Slow Code (Weeks 12–13)

For both CSC 1103 delivery (laptop) and CSC1104 delivery (Pi)

1) **Week 12 (dynamic memory).** Replace the fixed-size array with one allocated by malloc to fit the file, and compare memory use.
2) Replay the trades one at a time in timestamp order: at real speed for at least 10 minutes of trading, and through the whole of stress_full.csv at 100× and 1000× (a full day at 1000× takes under 2 minutes).
3) Show each signal on the Pi's built-in status LED (green for BUY, red for SELL) and on a coloured terminal display. (CSC1104 only)
4) Measure the processing latency of every trade and report the average and worst case.
5) Generate the assembly for your main loop (gcc -S -O2) and annotate what each instruction does.
6) **Show that slow code loses money.** Run the rival bot simulation below for Version 0 and your fastest version, and report how much money the slow code loses.

However, a fast program that gives wrong answers is worthless in trading. Every version on your ladder must still pass the test20.csv check.

The Rival Bot: Why Slow Code Loses Money

In real trading, you do not lose money by being slower than the market; you lose it by being slower than a rival who sees the same opportunity. When a signal appears, the fastest trader gets the current price, and everyone slower gets a later, usually worse, price. You will simulate this with an imaginary rival bot.

1) **Rival reaction time.** The rival reacts to every signal in R microseconds. Use R = 1 μs as the standard value.
2) **Your reaction time.** For each signal, your reaction time is the processing time of the trade that triggered it, measured with now_ns().
3) **Who gets the price.** If your reaction time is shorter than R, you win and trade at the price of the triggering trade. If it is longer, the rival wins and you trade at the price of the next trade in the file.
4) **Positions.** Start with no Bitcoin. On a BUY signal, buy 1 BTC if you hold none; on a SELL signal, sell it if you hold one. Your profit or loss (P&L) is the total of all selling prices minus all buying prices.
5) **Money lost to slow code.** Both versions raise exactly the same signals, so any difference in P&L comes purely from speed:

$$money\ lost = P\&L\ of\ fastest\ version - P\&L\ of\ Version\ 0$$

6) **Sweep the rival.** Repeat for R from 0.1 μs to 100 μs and plot money lost against R. Explain the point where each version starts losing.

This is a simplified simulation for learning about speed. It involves no real trading, ignores trading fees, and is not trading advice. The crossover strategy itself may lose money even when fast: the lesson is the gap between fast and slow code, not whether the strategy is profitable.

The Race Demonstration

Your demo video must include a race: Version 0 and your fastest version replay the same data side by side at an accelerated speed, in two terminal windows, each with an on-screen counter showing how far it has fallen behind the replay clock. Your fastest version also drives the built-in LED (for CSC1104). Each window also shows its running P&L against the rival bot. Explain in your own words why one keeps up and the other does not.

Stretch Goals (up to +10 marks in total, capped at 100)

These are optional, for groups who want to go further:

- **Hand-written assembly.** Re-implement one kernel (e.g. the VWAP sum of price × qty) in AArch64 assembly or NEON SIMD, called from C. Beat the compiler's -O3 version or explain precisely why the compiler already won.
- **Struct-of-Arrays.** Store each field in its own array instead of an array of structs, and explain any speedup using cache lines.
- **Percentile latency.** Report the median (p50) and 99th percentile (p99) latency, which requires sorting.
- **C versus Python.** Python is not taught in this course. Groups who learn it independently may write their own Python version of Stage A, run it on stress_full.csv, and explain why C is faster (and why NumPy comes closer).

The Maths You Need

This section gives every formula your C program must compute, with a small worked example for each. Work through the examples by hand before you write the code: if your program cannot reproduce them, it will not match test20_expected.txt either.

Symbols

Symbol	Meaning	From the file
t	Trade number, counting 1, 2, 3, … in file order	Line number
p_t	Price of trade t	Field 2 (price)
q_t	Quantity of trade t	Field 3 (qty)
τ_t	Timestamp of trade t, in microseconds	Field 5 (time)
N	Window size: how many recent trades an average uses	Your group's short or long window
M	Number of trades processed so far	Your counter

1. Volume-Weighted Average Price (VWAP)

The average price paid over all trades so far, where bigger trades count more:

$$VWAP = \frac{p_1 q_1 + p_2 q_2 + \ldots + p_M q_M}{q_1 + q_2 + \ldots + q_M}$$

Keep two running totals, updated once per trade: the sum of $p_t \times q_t$ and the sum of q_t. VWAP is one divided by the other.

Example. Three trades: 100 × 2, 101 × 1, 102 × 1. VWAP = (200 + 101 + 102) / (2 + 1 + 1) = 403 / 4 = **100.75**. The plain average of the prices would be 101; VWAP is lower because the biggest trade was at 100.

2. Simple Moving Average (SMA)

The plain average of the last N prices, at trade t:

$$SMA_N(t) = \frac{p_{t-N+1} + \ldots + p_{t-1} + p_t}{N}$$

The SMA only exists once at least N trades have arrived (t ≥ N). Compute it twice, once with your short window and once with your long window.

Example (Figure 1, trade 8). Short window N = 3: (101.0 + 101.6 + 102.1) / 3 = 304.7 / 3 = **101.57**. Long window N = 5: (100.9 + 101.3 + 101.0 + 101.6 + 102.1) / 5 = 506.9 / 5 = **101.38**.

3. Running-Sum SMA (the fast way)

Re-adding all N prices for every trade is slow when N is large. Instead, keep the window sum S_t and update it with one addition and one subtraction:

$$S_t = S_{t-1} + p_t - p_{t-N}$$

$$SMA_N(t) = \frac{S_t}{N}$$

Here p_t is the newest price joining the window and p_{t-n} is the oldest price leaving it. To know which price leaves, store the last N prices in an array of size N and overwrite the oldest one each time (a circular buffer).

Example (Figure 1, trade 9, price 101.8). Short window: S_9 = 304.7 + 101.8 − 101.0 = 305.5, so $SMA_3(9)$ = **101.83**. Long window: S_9 = 506.9 + 101.8 − 100.9 = 507.8, so $SMA_5(9)$ = **101.56**. Check by adding the windows directly: the answers are the same.

4. Crossover Signal

Take the difference between the short and long averages at each trade:

$$d_t = SMA_{short}(t) - SMA_{long}(t)$$

- **BUY** at trade t when $d_{t-1} \le 0$ and $d_t > 0$ (the short average has just risen above the long one).
- **SELL** at trade t when $d_{t-1} \ge 0$ and $d_t < 0$ (the short average has just fallen below the long one).
- No signal otherwise. The first possible signal is at trade N_long + 1, because you need d for two trades in a row.

Example. If d_6 = −0.70 and d_7 = +0.35, trade 7 is a BUY. If d_{17} = +0.29 and d_{18} = −0.15, trade 18 is a SELL.

5. One-Second OHLC Bars

Group trades by the whole second they happened in:

$$second = \frac{\tau_t}{1{,}000{,}000}$$

Use integer division, so the remainder (the microseconds within the second) is dropped.

For each second: **Open** is the price of the first trade, **High** the highest price, **Low** the lowest price, and **Close** the price of the last trade. A new bar starts when the second changes.

Example. Four trades in the same second at 100.2, 100.9, 100.1, 100.5 give Open = 100.2, High = 100.9, Low = 100.1, Close = 100.5.

6. Buy or Sell Side

is_buy = 1 if isBuyerMaker is False; is_buy = 0 if isBuyerMaker is True

7. Fixed-Point Numbers

Store a decimal as a whole number by multiplying by a fixed scale factor k, and divide by k only when printing:

$$stored\ value\ =\ decimal\ value\ \times\ k$$

Example. With k = 100 for price and k = 1,000,000 for quantity: 65012.34 is stored as 6501234, and 0.0015 is stored as 1500. Their product 6501234 × 1500 = 9,751,851,000 carries a combined scale of 100 × 1,000,000 = 10^8, so the value is 9,751,851,000 / 10^8 = **97.51851** USDT, matching quoteQty.

Watch for overflow. A long long holds up to about 9.2 × 10^{18}. If you scale both price and quantity by 10^8, one product can reach about 6.5 × 10^{12} × 10^8 = 6.5 × 10^{20}, far too big. Choose scales so that every product fits.

8. Timing, Speedup and Latency

$$time\ per\ trade\ =\ \frac{total\ run\ time\ (ns)}{number\ of\ trades}$$

$$Speedup_k\ =\ \frac{time\ of\ Version\ 0}{time\ of\ version\ k}$$

$$latency_t\ =\ time\ signal\ decided\ -\ time\ trade\ t\ released$$

$$average\ latency\ =\ \frac{latency_1\ +\ \ldots\ +\ latency_M}{M}$$

Example. If Version 0 takes 40 s and your version 4 s, Speedup = 40 / 4 = **10×**. On 3 million trades, 4 s is 4,000,000,000 / 3,000,000 ≈ **1,333 ns per trade**.

9. Replay Timing

$$release\ time\ of\ trade\ t\ =\ \frac{\tau_t\ -\ \tau_1}{speed}$$

At speed = 1000, trades that were 1 second (1,000,000 μs) apart in the market are released 1 ms apart.

10. Profit and Loss (Rival Bot)

$$P\&L\ =\ (sum\ of\ all\ selling\ prices)\ -\ (sum\ of\ all\ buying\ prices)$$

$$money\ lost\ to\ slow\ code\ =\ P\&L\ of\ fastest\ version\ -\ P\&L\ of\ Version\ 0$$

Example. Buy at 65,000.00, sell at 65,040.00, buy at 64,990.00, sell at 65,010.00: P&L = (65,040 + 65,010) − (65,000 + 64,990) = **+60.00** USDT.

General Steps (Just a Reference)

The steps below serve as a guideline, not a limit, for the five stages.

1) **Input.** Start with the helper library (Stage A), then your own read_trade() (Stage A+), then load everything into an array of structs (Stage B). A starting struct is:

```
typedef struct {
    long long ts_us;   // timestamp, microseconds
    long long price;   // fixed-point, e.g. price x 100
    long long qty;     // fixed-point, e.g. qty x 1000000
    int       is_buy;  // 1 = buy (isBuyerMaker is False), 0 = sell
} Trade;
```

2) **Fixed-point conversion.** Choose a scaling factor that suits the dataset's price and quantity ranges. Check that price × qty cannot overflow long long (maximum about 9.2 × 10^18), and explain your choice in the report.
3) **Indicators.** Write a separate function for each indicator. For trades i = 1 … M with price p_i and quantity q_i, VWAP is:

$$VWAP = \frac{\sum(p_i \times q_i)}{\sum q_i}$$

The simple moving average over the last N trades is the sum of the last N prices divided by N. The running-sum version keeps the window sum S up to date in one step:

$$S(new) = S(old) + newest\ price - oldest\ price\ in\ the\ window$$

Keep the last N prices in an array of size N, and overwrite the oldest one each time (a circular buffer). For OHLC bars, group trades by whole second (time / 1,000,000) and record the first, highest, lowest and last price in each second.

4) **Signal.** Let $s = SMA_{short} - SMA_long$. A BUY signal occurs when s changes from ≤ 0 to > 0; a SELL signal when s changes from ≥ 0 to < 0.
5) **Timing and speedup.** Use the provided now_ns() function for all timing. Run every measurement at least 5 times and report the middle (median) value. The speedup of version k is:

$$Speedup_k = \frac{time\ of\ version\ 0}{time\ of\ version\ k}$$

6) **Replay.** Trade i is released at wall-clock time $(time_i - time_k)/speed$ after the start. Its latency is the time from release to signal decision. The lag counter is how far your program is behind the replay clock.
7) **Signal output.** Call led_init() once at the start, then use led_on(LED_GREEN), led_off(LED_GREEN), led_on(LED_RED) and led_off(LED_RED), and call led_cleanup() at the end. On the Pi, run with sudo; on a laptop, the LED state is printed instead. Print each signal in colour on the terminal using ANSI escape codes (e.g. \033[32m for green, \033[31m for red). Reboot when finished to restore the LED's normal behaviour.
8) **Plotting.** Open the output file once before your loop, write one line per trade inside it (only once both SMAs exist), and close it after the loop. Each line holds: trade number, price, short SMA, long SMA and signal (1 = BUY, −1 = SELL, 0 = none):

```
FILE *out = fopen("signals.dat", "w");      /* before the loop */
fprintf(out, "%d %.2f %.4f %.4f %d\n",
        t, price, sma_short, sma_long, signal); /* inside the loop */
fclose(out);                                  /* after the loop  */
```

Then run gnuplot plot_signals in the same folder to create signals.png. Use FILE * the same way you use the helper library for now; you will learn what it means in Week 10. Never write plot files in the versions you time: writing to a file is slow and would distort your measurements.

9) **Hardware counters.** Use perf stat to report cycles, instructions, instructions per cycle (IPC) and cache misses for your main versions.
10) **Prediction logbook.** Before every measurement, write down your predicted result and your reason. After measuring, record the actual result and explain any difference.
11) **Comment** on the differences across your versions and on what each change taught you about the hardware.

General Instruction

The programming language to be used is standard C. The GNU Compiler Collection (GCC) is the standard compiler, and all final measurements must be taken on the Raspberry Pi 5 issued to your group. Microsoft Visual Studio Code (VSC) is the baseline editor; you may use its Remote-SSH extension to edit code directly on the Pi.

You may also develop on your own laptop. Set it up once as follows:

Computer	Set-up
Windows	Install VS Code with the C/C++ extension, and the GCC compiler. Check with gcc --version in a terminal. Install gnuplot from gnuplot.info (Windows installer).
Mac	Install VS Code with the C/C++ extension, then run xcode-select --install in Terminal. On a Mac, the gcc command runs Apple's clang compiler, which is fine for this course. Install gnuplot with Homebrew: brew install gnuplot.
Raspberry Pi 5	Already set up: gcc, make, gdb, valgrind, perf and gnuplot are installed.

perf, taskset and the -mcpu=cortex-a76 flag work only on the Pi. On a laptop, use the plain -O0, -O2 and -O3 flags.

The following are provided in the SIT-Xsite Content and Dropbox folders under the course assessment folder:

Provided	What it is
timing.h	now_ns(): current time in nanoseconds; sleep_ms(): pause. Works on Windows, Mac and Pi
led.h	led_init(), led_on(), led_off(), led_cleanup(). Drives the Pi's built-in LED; prints the LED state on Windows and Mac
test_helpers.c	A small program that checks timing.h and led.h work on your computer
trade_io.h / trade_io.c	Helper library: open_trades(), next_trade(), trade_price() etc.
Dataset files	test20.csv, dev_1M.csv (per group) and stress_full.csv
Expected answers	Correct results for your group's test20.csv and window sizes

Each group is issued the following kit:

Item	Note
Raspberry Pi 5 (4GB)	Raspberry Pi OS 64-bit, pre-installed; its built-in status LED is used for the signal
microSD card	Holds the OS and your work;

The tools gcc, make, gdb, valgrind, perf and gnuplot (http://www.gnuplot.info) are installed on the Pi. Use gnuplot for the Stage A signal plot. The other graphs in your report (speedup, latency, money lost) may be drawn with any tool, such as gnuplot, Excel or Google Sheets;

they are marked on what they show, not on how they look. Pin your program to one CPU core (taskset) and set the CPU governor to performance when benchmarking.

Use of Generative AI must follow the University rules

(https://www.gla.ac.uk/myglasgow/sld/ai/students/#academicintegrity). Every group member must be able to explain any line of the group's code during the live demonstration if schedule permits.

What to Hand In

According to your group number, use SIT-Xsite Dropbox to submit a single zip file with the contents of UoG-PMCOS, plus a separate PMCOSReportGroupxx.pdf file and the scanned signed declaration forms of all members in the group. Peer evaluation form

To aid in testing and assessing your code, please make sure that:

1) Your submission file is named UoG-PMCOA.zip.
2) When uncompressed, your files are in a folder named UoG-PMCOA_Groupxx. The folder should contain:
   - all C source code (.c, .h) for every stage, any assembly files (.S) and a Makefile, so that typing make on a Raspberry Pi 5 builds every version on your ladder;
   - a README with build and run instructions;
   - all printed results, benchmark logs and perf outputs, and your signals.dat and signals.png;
   - PMCOSReportGroupxx.pdf, where xx is your group number according to the grouping list;
   - a group PowerPoint presentation video of about 15 minutes (about 3 minutes per member), with cameras on, where each member presents their own portion of work (use the template provided);
   - a demo video of about 5 minutes showing your program running on the laptop and Pi, including the race demonstration and the LED signal;
   - the signed declaration form of each member.
3) Your PMCOAReportGroupxx.pdf file should contain:
   - the names of group members and each member's individual contribution on the cover page (see attached template);
   - your design solution: problem definition, problem analysis and pseudocode;
   - your stages' signal plot;
   - your speedup ladder table and graph, latency results and perf results;
   - your prediction logbook;
   - your annotated assembly listing;
   - your rival bot results: P&L of each version and the money-lost graph;
   - comparison comments across versions;
   - any interesting aspects of your solution (e.g. assumptions made, optimisations you thought of);
   - all source code as appendices.

4) Your submission will be tested for Turnitin plagiarism. Please refer to the guideline on https://www.gla.ac.uk/myglasgow/policy/regulations-current/general/plagiarism/ and the rules on the use of Gen AI. Plagiarism cases will be dealt with case by case, but there will be little tolerance.

5) It is highly recommended to use Overleaf for writing the report, as it is useful for collaborative writing. It is free for University of Glasgow students (https://www.overleaf.com/for/universities).
6) Peer evaluation form will be submitted separately

How This Exercise Will Be Marked

Following timely submission, the exercise will be given a numerical mark between 0 (no submission) and 100 (perfect in every way). The numerical marks will then be converted to a grade.

Scope-Level Weightage (Group)

Component	Weight
Stage 0: Pi set-up, first timings, LED blink	5%
Stage A: Streaming engine, correct on test20.csv, with signal plot	25%
Stage A+: Own reader with pointers and fast string parser	10%
Stage B: In-memory engine and efficiency ladder	10%
Stage C: malloc, replay, LED/terminal signal, annotated assembly, rival bot P&L	10%
Testing and Enhancement (below)	30%
Stretch goals	up to +10 (capped at 100)

Testing and Enhancement (30%)

1) **Performance measurement on the Laptop (CSC1103) & Pi (CSC1104) (10%).** Each member runs and documents the full benchmark on stress_full.csv on the Laptop (CSC1103) and Pi (CSC1104), **one set of documentation per member**. Documentation includes:
   - the system set-up (Pi model, OS, compiler version, flags, CPU governor);
   - timing and throughput (trades per second) for every version;
   - perf stat results (cycles, instructions, IPC, cache misses);
   - memory usage for each version.
2) **Prediction logbook (5%).** At least six predict–measure–explain entries, with at least one per stage, each dated and signed by the member who made it.
3) **Performance discovery (15%).**
   - Speedup ladder table and graph against Version 0, all measured on stress_full.csv.
   - Average and worst-case latency at each replay speed.
   - The race demonstration, showing the lag counter and P&L for Version 0 and your fastest version.
   - Money lost to slow code: P&L of each version against the rival bot, and the graph of money lost against the rival's reaction time.
   - Analysis of trade-offs, e.g. speed versus code readability, fixed-point precision versus double.
   - Recommendation of which version to deploy on a power- and memory-limited edge device, and why.

Overall Marking Criteria

- **30 marks** for the quality and structure of the code. Use appropriate names for variables and functions, and structure your source code into modules properly.
- **20 marks** for comments and documentation in the code and in PMCOAReportGroupxx.pdf. Comment the basic steps of each function and the variables used. Do not make an essay of your code; use your pdf file to discuss further details.
- **30 marks** for a solution that produces correct results. Partial marks will be awarded if your solution is on the right track but produces incorrect results due to minor errors. Marks will be awarded based on the originality of the code, which must be produced by the team.
- **20 marks** for optimisations and their evidence: measured speedups, lower latency, efficient memory use, and the quality of your explanations of why each change worked on the laptop and Pi's hardware.

Individual Mark

Each group member's grade = weighted peer review score × Group score (Group Project Report + Group Demo Video)+ presentation in the group video.